

Proyecto 1: Normalizador Estadístico Vectorizado en Ensamblador x86-64

Andrés Guillén Muñoz, a.guillen.m04@estudiantec.cr
Matías Camacho Abarca, jecamacho@estudiantec.cr

Resumen—Se implementan dos versiones funcionalmente equivalentes de un normalizador estadístico sobre arreglos de punto flotante de precisión simple, escritas íntegramente en ensamblador NASM x86-64: una escalar y una vectorizada con AVX2 que procesa ocho elementos por iteración. Ambas se enlazan a un mismo programa principal en C que respeta la ABI System V AMD64. La correctud se verifica contra una referencia en Python/NumPy sobre once casos borde y el rendimiento se mide con `clock_gettime` y RDTSC sobre cuatro escalas de tamaño. El *speedup* medido decrece de $7,70\times$ con $N = 10^3$ a $6,02\times$ con $N = 5 \times 10^7$, y los contadores de hardware explican la brecha: la versión vectorial ejecuta entre 7,6 y 7,9 veces menos instrucciones —prácticamente el límite teórico de $8\times$ que impone el ancho de los registros YMM— pero consume solo entre 4,7 y 5,5 veces menos ciclos, diferencia atribuible al ancho de banda de memoria. Se documenta además un resultado numérico relevante: la acumulación secuencial en `float32` con un único acumulador excede la tolerancia exigida a partir de $N = 10^6$, y se resuelve con suma compensada de Kahan, que alcanza un error relativo de $2,74 \times 10^{-9}$ con $N = 5 \times 10^7$ sin abandonar la precisión simple.

Palabras clave—SIMD, AVX2, NASM, x86-64, vectorización, reducción horizontal, System V AMD64 ABI, ancho de banda de memoria, precisión en punto flotante, suma compensada de Kahan.

I. INTRODUCCIÓN Y OBJETIVOS

El modelo de ejecución tradicional de un procesador es SISD (*Single Instruction, Single Data*): cada instrucción aritmética opera sobre un único dato. Las extensiones vectoriales introducen el modelo SIMD (*Single Instruction, Multiple Data*), en el que una sola instrucción opera simultáneamente sobre varios datos empaquetados en un registro ancho. En AVX2 los registros YMM son de 256 bits, de modo que una instrucción como `vaddps` procesa ocho valores de `float32` a la vez, frente al único valor de su equivalente escalar `addss`.

La ganancia no proviene de aumentar la frecuencia del reloj ni el número de núcleos, sino de reducir el número de instrucciones necesarias para el mismo trabajo, aprovechando el paralelismo de datos que existe de forma natural en problemas como el procesamiento de señales, el álgebra lineal o la estadística sobre grandes volúmenes.

Este trabajo implementa un **normalizador estadístico**: dado un arreglo de N valores en `float32`, calcula su suma, media, varianza, desviación estándar, mínimo y máximo, y produce el arreglo normalizado $y_i = (x_i - \mu) / \sigma$. El cálculo se implementa dos veces, de forma funcionalmente equivalente:

- Una versión **escalar**, que usa únicamente instrucciones de un solo dato (`movss`, `addss`, `minss`, `divss`, ...);

- Una versión **vectorizada con AVX2**, que procesa ocho elementos por iteración e incluye manejo explícito del remanente cuando N no es múltiplo de ocho.

Ambas se escriben íntegramente en NASM y se enlazan como funciones externas a un *driver* en C que se encarga de la entrada/salida, la reserva de memoria alineada y la medición de tiempos. No se emplean intrínsecos de C ni autovectorización del compilador en ningún punto del proyecto.

Los objetivos cuantitativos son: (i) verificar que ambas versiones producen resultados equivalentes dentro de la tolerancia relativa de 10^{-4} exigida, incluyendo los casos borde; (ii) medir el *speedup* sobre cuatro escalas de N ; y (iii) explicar, con contadores de hardware, por qué el *speedup* real se aparta del $8\times$ teórico.

El documento se organiza como sigue. Los diagramas de arquitectura se entregan como archivo independiente, `Diagramas_P1_Arquitectura.pdf`, y se describen en la sección III. La sección II describe el entorno de pruebas. Las secciones IV y V explican cada implementación y justifican sus decisiones de diseño. La sección VI presenta la verificación de correctud y la VII los resultados de rendimiento. La sección VIII documenta la inspección de bajo nivel con GDB y la IX el análisis de precisión numérica. La sección X cierra con las conclusiones.

II. ENTORNO DE PRUEBAS

Todas las mediciones se realizaron sobre el equipo y las versiones de herramientas de la Tabla 1. El soporte de AVX2 se verificó antes de iniciar el desarrollo.

Tabla 1: Entorno de desarrollo y medición.

Elemento	Valor
Equipo	Lenovo ThinkPad T460
Procesador	Intel Core i5-6300U @ 240 GHz
Microarquitectura	Skylake (familia 6, modelo 78, rev. 3)
Núcleos / hilos	2 físicos / 4 lógicos (Hyper-Threading)
Frecuencia	0,4 a 3,0 GHz (base 2,4 GHz)
Caché L1d / L1i	32 KiB por núcleo, 8 vías
Caché L2	256 KiB por núcleo, 4 vías
Caché L3	3 MiB compartida, 12 vías
Línea de caché	64 bytes
Extensiones SIMD	SSE-SSE4.2, AVX, AVX2, FMA
TSC	<code>constant_tsc</code> (frecuencia fija)
Sistema operativo	Ubuntu 24.04 LTS
NASM	2.16.01
GCC	13.3.0
GDB	15.1

La confirmación del soporte se obtuvo con:

```
1 $ lscpu | grep -o avx2
2 avx2
```

Listing 1: Verificación del soporte de AVX2.

Dos datos de la salida de `lscpu` tienen consecuencias directas sobre el diseño. Primero, **no aparece ningún indicador `avx512*`**: el hardware no soporta AVX-512, lo que determina la elección de extensión discutida en la sección V-A. Segundo, sí aparece `fmavt`, de modo que las instrucciones de multiplicación y suma fusionadas estaban disponibles y su descarte responde a un motivo técnico y no a una limitación del equipo.

II-A. Opciones de compilación

El proyecto se compila con el Makefile entregado. Los kernels se ensamblan con `nasm -f elf64 -g -F dwarf` y el *driver* con `gcc -std=gnu11 -Wall -Wextra -O2 -g`. La información de depuración DWARF que genera NASM es la que permite poner puntos de ruptura sobre las etiquetas del ensamblador en la sección VIII.

No se emplean intrínsecos de C ni banderas de autovectorización en ningún punto del proyecto. En particular, la lectura del contador de marcas de tiempo del procesador se implementa con **ensamblador en línea** y no con el intrínseco `__rdtsc()` de `<x86intrin.h>`, de modo que el proyecto no contiene ningún intrínseco de compilador ni siquiera fuera de los kernels evaluados.

II-B. Protección de pila no ejecutable

NASM no emite por defecto la sección `.note.GNU-stack`, que GCC sí genera. Ante un objeto que no declara esa sección, el enlazador adopta la hipótesis conservadora de que el código podría necesitar una pila ejecutable y **desactiva la protección NX del ejecutable completo**. Un solo objeto de NASM degrada así la seguridad de todo el binario. Se añadió al final de ambos archivos:

```
1 section .note.GNU-stack noalloc noexec nowrite progbits
```

Listing 2: Declaración explícita de pila no ejecutable.

La verificación se hace sobre el binario enlazado:

```
1 $ readelf -l bin/norm_scalar | grep -Al GNU_STACK
2 GNU_STACK 0x0000000000000000 ...
3 0x0000000000000000 ... RW 0x10
```

Los indicadores RW (sin E) confirman que la pila es legible y escribible pero no ejecutable.

III. DIAGRAMAS DE ARQUITECTURA

Los diagramas se entregan en el archivo independiente `Diagramas_P1_Arquitectura.pdf` (carpeta `docs/` del repositorio), elaborado a partir del código de `stats_scalar.asm` y `stats_vector.asm`. La Tabla 2 indica el contenido de cada página.

Tabla 2: Contenido del archivo de diagramas.

Pág.	Contenido
1	Bloques: <code>driver</code> , <code>C</code> , <code>stats_scalar.asm</code> y <code>stats_vector.asm</code> ; ABI System V (argumentos, <i>callee-saved</i>) y marco de pila de <code>compute_stats</code>
2	Flujo de control de <code>sum_array</code> , escalar y vectorial
3	Flujo de control de <code>compute_stats</code> (dos pasadas, Kahan, mín/máx, remanente)
4	Flujo de control de <code>normalize_array</code> (camino temporal/no temporal, remanente)
5	Tabla de asignación de registros de los tres kernels
6	Mapa de memoria: alineación a 32 B, avance de 4 B contra 32 B y remanente

IV. IMPLEMENTACIÓN ESCALAR

IV-A. Convención de llamada

Las tres funciones respetan la ABI System V AMD64 [1]: los argumentos enteros y punteros viajan en `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`; los flotantes en `xmm0`–`xmm7`, en una cola independiente; el valor de retorno flotante en `xmm0`. El *callee* debe preservar `rbx`, `rbp` y `r12`–`r15`; el resto, incluidos **todos** los registros vectoriales, es de libre uso. Además `rsp` debe ser múltiplo de 16 justo antes de cada `call`.

Aplicado a `compute_stats`, esto plantea un problema concreto: la función recibe **seis** argumentos que deben sobrevivir al cálculo y solo hay **cinco** registros *callee-saved*. La versión escalar copia cuatro a registros preservados y apila además `r8` y `r9` (los punteros a mínimo y máximo):

```
1 compute_stats:
2     push    rbx
3     push    r12
4     push    r13
5     push    r14
6     push    r15
7     push    r8           ; min* -> [rsp+8]
8     push    r9           ; max* -> [rsp]
9     test    esi, esi
10    jle     .zero_case
```

Listing 3: Prólogo escalar de `compute_stats`: siete `push`.

Al entrar a la función `rsp` $\equiv$ 8 (mód 16), porque el `call` empujó ocho bytes. Siete `push` restan 56 bytes: $8 - 56 = -48 \equiv 0$ (mód 16), de modo que `rsp` queda alineado. Se preserva `rbx` aunque esta versión no lo usa; además completa un número impar de `push`, necesario para la alineación. Un número par habría violado la ABI. En el camino `.zero_case` los punteros se escriben directamente desde `rdx`, `rcx`, `r8`, `r9`, que aún no se han modificado.

```
1     push    rbx
2     push    r12
3     push    r13
4     push    r14
5     push    r15           ; 5 pushes: rsp queda
6     test    esi, esi
7     jle     .cs_zero
8     sub     rsp, 16       ; espacio para el sexto
9     mov     [rsp], r9     ; [rsp] = puntero a max
```

Listing 4: Prólogo de `compute_stats`: cinco registros preservados y el sexto argumento en la pila.

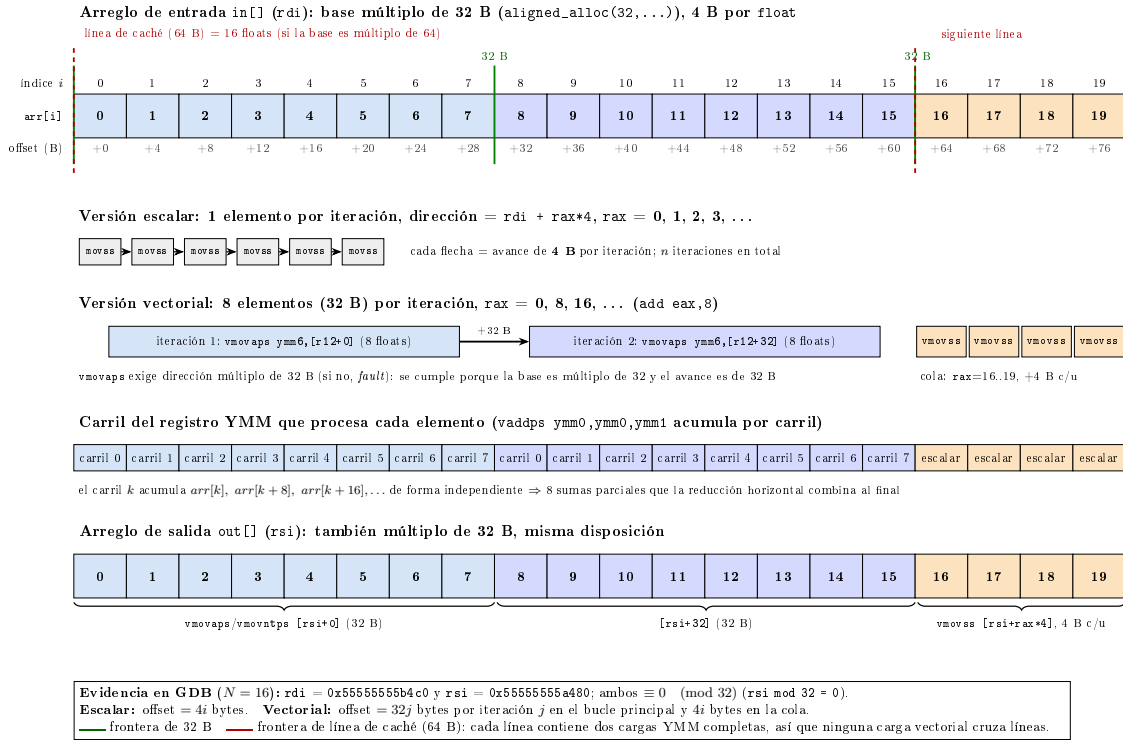
Disposición de memoria y avance del puntero (ejemplo con $N = 20$: $N \& \sim 7 = 16$)

Fig. 1: Mapa de memoria (página 6 de Diagramas_P1_Arquitectura.pdf).

La alineación merece comprobación explícita. Al entrar a la función `rsp` $\equiv 8$ (mód 16), porque el `call` del llamador empujó ocho bytes de dirección de retorno sobre una pila alineada. Cinco `push` restan 40 bytes: $8 - 40 = -32 \equiv 0$ (mód 16), y el `sub rsp, 16` no altera la congruencia. Un número par de `push` habría violado la ABI, con consecuencias impredecibles en cualquier función llamada que usara `movaps` sobre la pila.

IV-B. Cálculo de estadísticos: dos pasadas

La identidad $\text{Var}(X) = E[X^2] - E[X]^2$ permitiría calcular la varianza en un único recorrido, acumulando $\sum x_i$ y $\sum x_i^2$ a la vez. Se descartó por inestabilidad numérica; el análisis cuantitativo está en la sección IX-A. El algoritmo adoptado es de dos pasadas:

- recorrido para la suma, y de ahí $\mu = \sum x_i / n$;
- recorrido para $\sum (x_i - \mu)^2$, y de ahí $\sigma^2 = \sum (x_i - \mu)^2 / n$, fusionando en el mismo bucle el cálculo del mínimo y el máximo.

Fusionar `min` y `max` con la segunda pasada es deliberado: el bucle ya carga cada `arr[i]` desde memoria, y calcular los extremos en el mismo recorrido no cuesta ninguna lectura adicional, solo dos instrucciones aritméticas sobre un valor que ya está en un registro. Calcularlos por separado habría significado un tercer recorrido completo del arreglo.

```
1 .var_loop:
2   cmp     eax, r13d
3   jge     .var_done
```

```
4   movss   xmm3, [r12 + rax*4] ; xmm3 = arr[i]
5   movss   xmm6, xmm3
6   subss   xmm6, xmm0          ; d = arr[i] - mean
7   mulss   xmm6, xmm6          ; d^2
8   subss   xmm6, xmm7          ; y = d^2 - compensacion
9   movss   xmm8, xmm2
10  addss   xmm8, xmm6          ; t = suma + y
11  movss   xmm7, xmm8
12  subss   xmm7, xmm2
13  subss   xmm7, xmm6          ; c = (t - suma) - y
14  movss   xmm2, xmm8          ; suma = t
15  minss   xmm4, xmm3          ; min = min(min, arr[i])
16  maxss   xmm5, xmm3          ; max = max(max, arr[i])
17  inc     eax
18  jmp     .var_loop
```

Listing 5: Bucle de varianza con mínimo y máximo fusionados y suma compensada de Kahan.

Tres decisiones de diseño en este fragmento:

- Orden de las operaciones:** La resta $x_i - \mu$ se hace *antes* de elevar al cuadrado, de modo que la cantidad que se eleva es pequeña y nunca se construyen magnitudes enormes que después habría que cancelar. Esta es exactamente la diferencia con el método de una pasada.
- minss/maxss en lugar de comiss más salto condicional:** Ambas alternativas son correctas, pero `minss/maxss` resuelven la comparación y la selección en una sola instrucción y, sobre todo, **sin bifurcación**. Un `comiss` seguido de `jnb` introduciría un salto cuyo resultado depende de los datos; con datos aleatorios el predictor de saltos fallaría aproximadamente la mitad de las veces, y cada fallo cuesta entre 15 y 20 ciclos de penalización por vaciado del *pipeline*. Además estas

instrucciones se traducen una a una a `vmminps/vmaxps` en la versión vectorial.

- **Inicialización de los acumuladores de extremos con `arr[0]`:** Inicializarlos en 0,0 produciría un máximo de 0,0 en un arreglo íntegramente negativo, un valor que no está en los datos. Es seguro leer `arr[0]` porque el caso $n \leq 0$ se descarta antes.

El bloque central del bucle implementa la **suma compensada de Kahan**, cuya justificación se desarrolla en la sección IX-B. La idea es mantener un segundo acumulador (`xmm7`) con el error de redondeo perdido en la suma anterior y restárselo al siguiente término, de modo que el error no se acumule con n .

IV-C. Normalización

El caso degenerado $\sigma = 0$ ocurre cuando todos los valores son iguales; entonces el *z-score* no está definido y la especificación exige copiar la entrada sin normalizar. La comprobación se evalúa **una sola vez, fuera del bucle**:

```
1 normalize_array:
2     movss    xmm8, xmm0      ; mean, no lo pisa el bucle
3     movss    xmm9, xmm1      ; stddev
4     xorps    xmm10, xmm10
5     ucomiss   xmm9, xmm10     ; comparar stddev contra 0.0
6     je       .copy_loop      ; sigma == 0 -> copiar tal
7     ; cual
8     xor     eax, eax
9     .norm_loop:
10    cmp      eax, edx
11    jge      .norm_done
12    movss    xmm2, [rdi + rax*4]
13    subss    xmm2, xmm8      ; in[i] - mean
14    divss    xmm2, xmm9      ; / stddev
15    movss    [rsi + rax*4], xmm2
16    inc     eax
17    jmp      .norm_loop
```

Listing 6: Decisión de ruta según `stddev`, evaluada fuera del bucle.

Extracción de la condición del bucle (*branch hoisting*):

El valor de `stddev` es un argumento que no cambia durante la ejecución; comprobarlo dentro del bucle costaría n comparaciones y n saltos para obtener siempre la misma respuesta. El costo pasa de $O(n)$ a $O(1)$. La consecuencia estructural es que existen **dos bucles separados** en lugar de uno con una condición interna: se duplican unas pocas instrucciones de código a cambio de eliminar toda bifurcación dependiente de datos del camino crítico.

ucomiss y no comiss: Ambas comparan flotantes escalares y actualizan las banderas ZF, CF y PF. La diferencia está en el manejo de NaN: `comiss` lanza una excepción de operación inválida ante un QNaN, mientras que `ucomiss` (*unordered compare*) solo lo hace con un SNaN. Para una simple comprobación de igualdad no se desea que un NaN aborte el programa. Con $\sigma = \text{NaN}$ la comparación es no ordenada y activa ZF, PF y CF; el salto se toma y la entrada se copia sin normalizar, comportamiento aceptable para un dato inválido.

Comparación en punto flotante y no bit a bit: `ucomiss` contra 0,0 trata $-0,0$ y $+0,0$ como iguales, que es el comportamiento correcto: si σ llegara como $-0,0$, dividir seguiría siendo indefinido. Una comparación del patrón de bits

contra 0×00000000 no detectaría ese caso, porque $-0,0$ se representa como 0×80000000 .

El caso $n = 0$ **no necesita código aparte**: La condición del bucle se evalúa **antes** del cuerpo, de modo que con $n = 0$ el cuerpo no se ejecuta nunca. Es correcto por construcción y no por casualidad.

V. IMPLEMENTACIÓN VECTORIAL (AVX2)

V-A. Elección de AVX2 frente a SSE y AVX-512

Frente a SSE. SSE ofrece registros XMM de 128 bits, es decir cuatro `float32` por instrucción: la mitad del ancho de AVX2. Más importante que el ancho es la **codificación**: SSE usa instrucciones de dos operandos, en las que el destino es también un operando de entrada y se destruye. AVX introduce la codificación VEX de tres operandos, que permite escribir el resultado en un registro distinto de las fuentes. En el bucle principal esto es determinante:

```
1 vsubps    ymm7, ymm6, ymm3 ; ymm7 = ymm6 - ymm3,
   ymm6 INTACTO
```

El registro `ymm6` conserva los ocho valores originales y puede alimentar después a `vmminps` y `vmaxps`. Con SSE (`subps xmm6, xmm3`) el valor original se habría perdido y haría falta una copia explícita en cada iteración.

Frente a AVX-512. Los registros ZMM de 512 bits procesarían dieciséis `float32` por instrucción, el doble de AVX2. La razón del descarte es directa: **el procesador del equipo de pruebas no soporta AVX-512**, como confirma la ausencia de cualquier indicador `avx512*` en la salida de `lscpu`. Aun con soporte, la decisión no sería automática: en varias microarquitecturas de Intel el uso sostenido de instrucciones AVX-512 reduce la frecuencia del núcleo para mantenerse dentro del presupuesto térmico, de modo que el beneficio neto depende de la proporción de código vectorizado.

V-B. Marco de pila y alineación

En la versión vectorial `r8` y `r9` no se apilan por separado: `min*` va a `rbx` y solo `max*` necesita pila.

```
1 push    rbx
2 push    r12
3 push    r13
4 push    r14
5 push    r15 ; 5 pushes: rsp queda
6         alineado a 16
7 test    esi, esi
8 jle     .cs_zero
9 sub     rsp, 16 ; espacio para el sexto
         argumento
10 mov     [rsp], r9 ; [rsp] = max*
```

Listing 7: Prólogo vectorial de `compute_stats`: cinco `push` y `max*` en la pila.

Cinco `push` restan 40 bytes: $8 - 40 = -32 \equiv 0 \pmod{16}$, y `sub rsp, 16` no altera la congruencia. Los marcos de pila de ambas versiones se comparan en la página 1 de `Diagramas_P1_Arquitectura.pdf`.

V-C. Bucle principal

```

1 .cs_vec_loop:
2   cmp     eax, ecx
3   jge     .cs_reduce
4   vmovaps ymm6, [r12 + rax*4] ; 8 floats
5   vsubps  ymm7, ymm6, ymm3    ; x - mean
6   vmulps  ymm7, ymm7, ymm7    ; (x - mean)^2
7   vsubps  ymm7, ymm7, ymm11   ; termino corregido
   (Kahan)
8   vaddps  ymm8, ymm2, ymm7    ; nueva suma por carril
9   vsubps  ymm11, ymm8, ymm2
10  vsubps  ymm11, ymm11, ymm7   ; nueva compensacion
11  vmovaps ymm2, ymm8
12  vminps  ymm4, ymm4, ymm6     ; min por carril
13  vmaxps  ymm5, ymm5, ymm6     ; max por carril
14  add     eax, 8
15  jmp     .cs_vec_loop

```

Listing 8: Núcleo vectorizado: ocho elementos por iteración con tres acumuladores en paralelo.

El concepto central es que `ymm2`, `ymm4` y `ymm5` **no contienen un resultado, sino ocho resultados parciales independientes**. El carril k acumula los elementos $k, k+8, k+16, \dots$ de forma completamente separada de los demás. Al terminar el bucle hay ocho sumas parciales, ocho mínimos parciales y ocho máximos parciales que deben combinarse; ese es el propósito de la reducción horizontal. La sección VIII muestra este estado inspeccionado directamente en el registro con GDB.

Uso de `vmovaps` y no `vmovups`: La instrucción alineada exige que la dirección sea múltiplo de 32 y genera un fallo de protección si no lo es; la no alineada no lo exige pero puede ser más lenta cuando el acceso cruza una línea de caché. Su uso aquí es seguro porque el *driver* reserva la memoria con `aligned_alloc(32, ...)`, de modo que la dirección base es múltiplo de 32, y el índice `rax` avanza de ocho en ocho elementos, es decir exactamente 32 bytes por iteración. Toda dirección accedida en el camino principal conserva por tanto la alineación. La sección VIII incluye la verificación de esta propiedad en tiempo de ejecución. `sum_array` usa `vmovups` por ser la rutina de ejemplo provista; sobre memoria alineada su costo en Skylake es idéntico, y `compute_stats` y `normalize_array` usan `vmovaps`.

Truncamiento del contador: La instrucción `and ecx, ~7` calcula el mayor múltiplo de ocho menor o igual que n . En NASM `~7` es el complemento a uno de 7, es decir un patrón con los tres bits menos significativos en cero; el AND los borra, y como $2^3 = 8$ eso equivale a truncar al múltiplo de ocho inferior. Con $n = 20$ el resultado es 16. Es equivalente a $n - (n \bmod 8)$ pero en una sola instrucción de un ciclo, sin división.

V-D. Reducción horizontal

```

1 .cs_reduce:
2   vsubps  ymm2, ymm2, ymm11   ; incorporar el error de
   cada carril
3   vextractf128 xmm8, ymm2, 1  ; xmm8 = carriles 4-7
4   vaddps  xmm2, xmm2, xmm8    ; 4 sumas parciales
5   vhaddps xmm2, xmm2, xmm2    ; xmm2[0] = suma de los 8
6   vhaddps xmm2, xmm2, xmm2
7
8   vextractf128 xmm8, ymm4, 1
9   vminps  xmm4, xmm4, xmm8    ; 4 minimos parciales
10  vpermilps xmm8, xmm4, xmm8  ; xmm8[0..1] = xmm4[2..3]
11  vminps  xmm4, xmm4, xmm8    ; 2 minimos parciales

```

```

12  vpermilps xmm8, xmm4, 0x01 ; xmm8[0] = xmm4[1]
13  vminps  xmm4, xmm4, xmm8    ; xmm4[0] = minimo global

```

Listing 9: Reducción horizontal de los ocho carriles a un escalar.

Por qué hace falta `vextractf128`: AVX2 no implementa operaciones horizontales de 256 bits reales: `vhaddps` sobre un registro YMM opera de forma **independiente dentro de cada mitad de 128 bits** y nunca cruza la frontera entre ellas. Sumar los ocho carriles aplicando únicamente `vhaddps` es por tanto imposible, porque los elementos 0–3 y 4–7 nunca se encontrarían. La instrucción `vextractf128` extrae la mitad alta a un registro propio para que el `vaddps` siguiente pueda combinarlas. Esta restricción de carriles es una decisión de diseño del hardware, orientada a mantener cortas las rutas de datos internas, y constituye la fuente más frecuente de errores al vectorizar.

Motivo de uso de `vhaddps` en la suma y `vpermilps` en los extremos: Al no existir una instrucción de mínimo horizontal, el conjunto incluye `vhaddps` para la suma, pero no hay ningún `vhminps`. Para reducir mínimo y máximo no queda alternativa a permutar los carriles y volver a aplicar la operación vertical. Esta asimetría entre ambas reducciones no es una inconsistencia de estilo sino una consecuencia directa del conjunto de instrucciones disponible.

Decodificación del operando inmediato de `vpermilps`: El byte se interpreta como cuatro selectores de dos bits, del carril 0 al 3. Así, $0 \times 0E = 00\ 00\ 11\ 10$ produce los selectores $[2, 3, 0, 0]$, de modo que $\text{dst}[0] = \text{src}[2]$ y $\text{dst}[1] = \text{src}[3]$; combinado con el `vminps` siguiente, compara los carriles 0–1 contra los 2–3 y reduce de cuatro mínimos parciales a dos. Análogamente $0 \times 01 = 00\ 00\ 00\ 01$ produce $[1, 0, 0, 0]$ y el último `vminps` deja el mínimo global en el carril 0. La secuencia completa reduce $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.

Alternativa considerada. La instrucción `vhaddps` se decodifica en tres micro-operaciones, dos de ellas de mezcla, mientras que la secuencia `vshufps + vaddps` realiza la misma operación con menos trabajo [2]. Se conservó `vhaddps` por legibilidad, dado que la reducción se ejecuta **una vez por llamada** y no por iteración: con $N = 10^6$ son seis instrucciones frente a 125 000 iteraciones del bucle principal, de modo que su impacto en el tiempo total no es medible.

V-E. Manejo del remanente

Cuando N no es múltiplo de ocho, los últimos $N \bmod 8$ elementos no completan un vector y se procesan de uno en uno:

```

1 .cs_tail:
2   cmp     eax, r13d
3   jge     .cs_finish
4   vmovss  xmm6, [r12 + rax*4] ; un solo float
5   vminss  xmm12, xmm12, xmm6
6   vmasss  xmm13, xmm13, xmm6
7   vsubss  xmm6, xmm6, xmm15   ; x - mean
8   vmulss  xmm6, xmm6, xmm6
9   vsubss  xmm6, xmm6, xmm9    ; termino corregido
10  vaddss  xmm7, xmm2, xmm6
11  vsubss  xmm9, xmm7, xmm2
12  vsubss  xmm9, xmm9, xmm6
13  vmovss  xmm2, xmm7
14  inc     eax
15  jmp     .cs_tail

```


Listing 10: Bucle de cierre para los elementos sobrantes.

El bucle emplea las versiones escalares (*ss) de exactamente las mismas operaciones que el bucle vectorial (*ps), y el índice `eax` llega a él valiendo $n \& \sim 7$, es decir justo donde el bucle vectorial se detuvo.

Un detalle de diseño facilita la transición: **el registro `xmmk` es la mitad baja del registro `ymmk`**, de modo que los valores dejados por la reducción horizontal ya se encuentran donde el bucle escalar los necesita, sin ninguna instrucción de transferencia.

V-F. Transición AVX-SSE

Tras ejecutar instrucciones AVX, los 128 bits altos de los registros YMM quedan en un estado que el procesador debe preservar. Si el código que sigue ejecuta instrucciones SSE de 128 bits —como las que emite GCC al compilar el *driver*— las microarquitecturas de Intel desde Skylake deben insertar dependencias falsas o realizar una transición de estado con penalizaciones de decenas de ciclos por ocurrencia [3].

La instrucción `vzeroupper` pone en cero las mitades altas de todos los registros YMM y comunica al hardware que no hay estado AVX pendiente. Se coloca antes de **cada** `ret` de las funciones que usan registros YMM; la versión escalar no la necesita porque nunca toca un YMM.

V-G. Optimización de `normalize_array`

La medición por fases (sección VII) identificó `normalize_array` como la función con menor *speedup*, lo que motivó dos optimizaciones cuyo comportamiento resultó complementario.

Caso $\sigma = 0$ sin segundo bucle. A diferencia de la versión escalar, que duplica el bucle para copiar la entrada, la vectorial sustituye los parámetros: si `vucomiss` detecta $\sigma = 0$, asigna $\mu = 0,0$ y $\sigma = 1,0$, de modo que $(x - 0) \cdot (1/1) = x$ exactamente en IEEE-754 (restar cero y multiplicar por uno no redondean). Se evita duplicar el bucle vectorial y su variante no temporal a cambio de dos instrucciones, y la comprobación sigue fuera del bucle.

Multiplicación por el recíproco: La instrucción `vdivps` tiene latencia de unos 14 ciclos y, más relevante, un *throughput* recíproco de aproximadamente 5 ciclos, porque la unidad de división no está completamente segmentada; `vmulps` presenta 4 y 0,5 respectivamente [2]. Se calcula $1/\sigma$ una sola vez fuera del bucle y se multiplica dentro. El costo es un redondeo adicional, del orden de 10^{-7} relativo, despreciable frente a la tolerancia de 10^{-4} .

Almacenamiento no temporal: Al escribir con `vmovaps`, el procesador lee primero la línea de caché de destino para tomar posesión de ella (*read for ownership*) aunque vaya a sobrescribirla por completo, lo que desperdicia la mitad del ancho de banda efectivo. La instrucción `vmovntps` escribe directamente a memoria y evita esa lectura.

La Tabla 3 muestra que ambas optimizaciones son **mutuamente excluyentes**: el recíproco solo ayuda cuando el

Tabla 3: Aceleración de `normalize_array` según la optimización aplicada.

N	Solo recíproco	Recíproco + <code>vmovntps</code>
10^5	2,20×	0,97×
2×10^7	0,95×	1,62×

conjunto de trabajo cabe en caché, y `vmovntps` solo ayuda cuando no cabe, porque evita la caché por diseño.

La solución adoptada elige el camino **una sola vez fuera del bucle**, aplicando el mismo principio de *branch hoisting* que en la comprobación de $\sigma = 0$:

```

1  cmp     edx, NORM_NT_UMBRAL
2  jae     .norm_vec_loop_nt
3
4  .norm_vec_loop:                ; camino TEMPORAL
5  ...
6  vmulps  ymm2, ymm2, ymm9      ; (x - mean) * (1/stddev)
7  vmovaps [rsi + rax*4], ymm2
8  ...
9  .norm_vec_loop_nt:            ; camino NO TEMPORAL
10 ...
11 vmovntps [rsi + rax*4], ymm2
12 ...
13 .norm_tail_nt:
14 sfence                        ; ordena los
                               almacenamientos no temporales

```

Listing 11: Selección del camino de escritura según el tamaño del arreglo.

El umbral se deriva de un criterio físico: `normalize_array` toca ocho bytes por elemento (cuatro de entrada más cuatro de salida), de modo que su conjunto de trabajo es $8N$ bytes. Con una caché L3 de 3 MiB, el punto de quiebre está en $3 \text{ MiB} / 8 = 393\,216$ elementos.

La Tabla 4 valida el criterio experimentalmente. El camino temporal se degrada progresivamente conforme el conjunto de trabajo se aproxima a la capacidad de L3, y al cruzar el umbral el costo por elemento cae un 29% y se mantiene constante.

Tabla 4: Calibración empírica del umbral (200 repeticiones).

N	Conjunto (8N)	Camino	ns/elemento
100 000	800 KB	temporal	0,285
200 000	16 MB	temporal	0,475
300 000	24 MB	temporal	0,672
400 000	32 MB	no temporal	0,475
500 000	40 MB	no temporal	0,464
700 000	56 MB	no temporal	0,450
1 000 000	80 MB	no temporal	0,501

VI. CASOS DE PRUEBA

La verificación es automática. La herramienta `tools/run_tests.py` ejecuta la batería completa contra ambos binarios y compara además las dos salidas entre sí. El veredicto de cada caso se toma del **código de salida** de los verificadores y no de una búsqueda de texto en su salida, lo que elimina falsos positivos por coincidencias parciales.

Los estadísticos se contrastan contra una referencia calculada con NumPy en doble precisión. El arreglo normalizado se verifica elemento por elemento con el criterio mixto

$$|y_i - \hat{y}_i| \leq \text{atol} + \text{rtol} |\hat{y}_i|, \quad (1)$$

con $\text{rtol} = 10^{-4}$ y $\text{atol} = 10^{-6}$. Se emplea un criterio mixto y no puramente relativo porque los elementos cuyo valor de entrada cae muy cerca de la media producen un *z-score* prácticamente nulo, y ahí el error relativo se dispara aunque el error absoluto sea despreciable. Un caso concreto con $N = 100\,000$: el elemento 47437 vale $-0,19254661$ frente a una media de $-0,19206424$, de modo que su *z-score* es $-8,36 \times 10^{-6}$; su error absoluto es $7,61 \times 10^{-9}$ pero el relativo asciende a $9,10 \times 10^{-4}$. Solo 3 de 100 000 elementos presentaban este artefacto, y el peor error absoluto de todo el arreglo era $1,22 \times 10^{-5}$.

La Tabla 5 recoge los once casos. La Tabla 6 muestra el detalle numérico de un caso representativo.

Obsérvese que *min* y *max* presentan errores tres órdenes de magnitud menores que el resto. La razón es estructural: son operaciones de **selección** y no de acumulación, por lo que devuelven un valor que está exactamente en el arreglo de entrada y no arrastran error de redondeo alguno.

VII. RESULTADOS DE RENDIMIENTO

VII-A. Metodología de medición

El *driver* toma **cuatro marcas de tiempo por repetición**, de modo que cada fase del kernel se obtiene por diferencia:

$$t_a \rightarrow \text{sum_array} \rightarrow t_b \rightarrow \text{compute_stats} \rightarrow t_c \rightarrow \text{normalize_array} \rightarrow t_d$$

Emplear cuatro marcas en lugar de tres pares independientes reduce a la mitad el sobrecosto de *clock_gettime* y garantiza que el total sea exactamente la suma de las partes, sin huecos ni solapes.

El conteo de ciclos se obtiene con la instrucción *RDTSC*, encerrada entre dos barreras *LFENCE*. Las barreras son necesarias porque el procesador ejecuta fuera de orden y podría adelantar la lectura del contador antes de que el kernel termine, produciendo una medición corta. Un matiz importante para la interpretación: el indicador *constant_tsc* del procesador confirma que el TSC avanza a una **frecuencia fija de referencia** y no a la frecuencia real del núcleo, que varía entre 0,4MB y 3MB según la carga. El valor sirve por tanto para comparar ambas versiones entre sí, pero no es idéntico a los ciclos de núcleo que reporta *perf stat -e cycles*.

Cada medición consta de 30 repeticiones más una **iteración de calentamiento que no se mide**. La primera ejecución paga el costo de traer el arreglo a la caché desde memoria principal y de resolver los primeros fallos de TLB y de predicción de saltos; incluirla entre las muestras inflaba la desviación estándar hasta el doble de la media sin aportar información sobre el régimen estable.

Se reporta la desviación estándar **muestral** (divisor $n - 1$), porque las repeticiones constituyen una muestra de las ejecuciones posibles y no la población completa. Se adoptó como criterio de calidad repetir toda medición cuya desviación superara el 5 % de la media, lo que permitió descartar varias tandas contaminadas por actividad concurrente del sistema.

VII-B. Tiempos y speedup

El caso $N = 10^5$ se midió con 200 repeticiones en lugar de 30. Su desviación muestral permanecía elevada incluso con la

máquina en reposo, y el análisis lo explica: con un tiempo de kernel de $137\mu\text{s}$ por repetición, las interrupciones periódicas del sistema operativo (las cuales ocurren cada 1 a 4 ms) añaden decenas de microsegundos a unas pocas muestras, produciendo una distribución sesgada. El **error estándar del promedio**, $\sigma/\sqrt{n} = 0,0284/\sqrt{200} = 0,0020$ ms, corresponde al 15 % de la media, de modo que el promedio está bien determinado aunque las muestras individuales se dispersen.

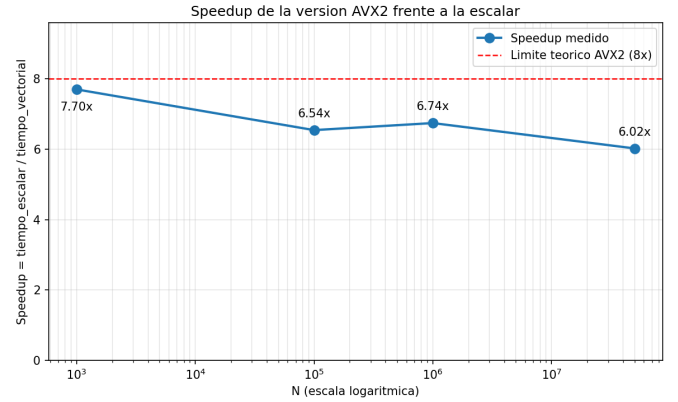


Fig. 2: *Speedup* de la versión AVX2 frente a la escalar en función de N (eje horizontal logarítmico). La línea punteada marca el límite teórico de 8× que impone el ancho de los registros YMM.

La Tabla 8 revela que *compute_stats* se mantiene entre 7,2 y 8,3× en todas las escalas, mientras que *normalize_array* cae de 7,75 a 3,06×. Esto sucede por dos motivos: *normalize_array* es la **única fase que escribe**, de modo que soporta el doble de tráfico de memoria que las demás, y su bucle es el más simple, con menos cómputo por byte transferido tras el que ocultar la latencia. Es un caso directo de la ley de Amdahl: la fase que menos se acelera termina dominando el tiempo total.

VII-C. Ciclos por elemento

Esta métrica es especialmente informativa porque normaliza por la cantidad de trabajo: es el costo de procesar un elemento, independiente de cuántos haya. La versión escalar se mantiene **prácticamente plana** (de 26,85 a 24,60), mientras que la vectorial alcanza su mínimo en $N = 10^5$ y después **crece un 19 %** hasta $N = 5 \times 10^7$ haciendo exactamente el mismo trabajo por elemento.

La asimetría identifica el cuello de botella de cada versión. La escalar está limitada por la **latencia** de su cadena de dependencias: cada *addss* debe esperar el resultado del anterior, y la compensación de Kahan alarga aún más esa cadena, de modo que salir de la caché apenas la afecta porque ya estaba esperando. La vectorial, mucho más rápida en cómputo, es la que sufre cuando los datos dejan de llegar a tiempo.

VII-D. Contadores de hardware

VII-E. Discusión: cómputo o ancho de banda

El conteo de instrucciones confirma el beneficio teórico de SIMD: La versión vectorial ejecuta entre 7,58 y 7,87

Tabla 5: Casos borde exigidos por la sección 2.3 del enunciado. Todos verificados contra la referencia en NumPy y entre ambas versiones.

N	Entrada	Qué verifica	Estad. escalar	Estad. vectorial	Escalar vs. vectorial
0	—	error controlado, sin división por cero	PASA	PASA	PASA
1	aleatoria	$\sigma = 0$, un solo elemento	PASA	PASA	PASA
7	aleatoria	$N < 8$: solo bucle remanente	PASA	PASA	PASA
8	aleatoria	exactamente un vector	PASA	PASA	PASA
15	aleatoria	N no múltiplo de 8	PASA	PASA	PASA
16	aleatoria	dos vectores completos	PASA	PASA	PASA
1000	constante (5,0)	$\sigma = 0$, ruta de copia	PASA	PASA	PASA
16	extremos	$\pm 10^6$, negativos, muy pequeños	PASA	PASA	PASA
1000	extremos	valores extremos a mayor escala	PASA	PASA	PASA
1000	aleatoria	caso general	PASA	PASA	PASA
100000	aleatoria	tamaño medio	PASA	PASA	PASA

Tabla 6: Detalle numérico del caso $N = 1000$, entrada aleatoria con semilla 42, versión escalar.

Campo	Referencia	Obtenido	Error rel.
sum	2512,394108	2512,393550	$2,22 \times 10^{-7}$
mean	2,512394	2,512393	$2,54 \times 10^{-7}$
var	3310,282586	3310,281010	$4,76 \times 10^{-7}$
stddev	57,535055	57,535042	$2,34 \times 10^{-7}$
min	-99,918816	-99,918816	$1,28 \times 10^{-10}$
max	99,981567	99,981567	$1,72 \times 10^{-10}$

Tabla 7: Tiempo del kernel completo, promedio $\pm$ desviación estándar muestral.

N	Escalar [ms]	Vectorial [ms]	Speedup
10^3	$0,0102 \pm 0,0001$	$0,0013 \pm 0,0000$	$7,70\times$
10^5	$1,0191 \pm 0,0465$	$0,1371 \pm 0,0284$	$7,44\times$
10^6	$9,9124 \pm 0,2005$	$1,4697 \pm 0,0579$	$6,74\times$
5×10^7	$492,75 \pm 0,80$	$81,81 \pm 0,37$	$6,02\times$

veces menos instrucciones que la escalar, muy cerca del límite de $8\times$ que impone el ancho de los registros YMM. La pequeña diferencia se explica por el bucle de remanente y la reducción horizontal, que son escalares. Es la métrica más limpia del beneficio de la vectorización porque **no depende del reloj, de la caché ni de la carga del sistema**; mide cuántas instrucciones hizo falta ejecutar para el mismo trabajo. En instrucciones por elemento son 38,83 frente a 4,94 con $N = 10^6$.

La brecha entre instrucciones y ciclos mide el costo de la memoria: Si el cuello de botella fuera puramente de cómputo, ambas razones coincidirían: ejecutar 7,87 veces menos instrucciones tomaría 7,87 veces menos ciclos. Los ciclos, sin embargo, solo caen $4,69\times$ con $N = 10^6$ y $5,50\times$ con $N = 5 \times 10^7$. Se pierde así el 40% y el 27% del beneficio potencial respectivamente, y esa pérdida es el tiempo que el procesador pasa esperando a la memoria en lugar de calculando. **Esta es la explicación cuantitativa de por qué el speedup real no alcanza el $8\times$ teórico.**

El IPC menor de la versión vectorial no indica ineficiencia: El IPC cuenta instrucciones completadas por ciclo, pero no todas las instrucciones realizan el mismo trabajo, un `vaddps` procesa ocho valores y un `addss` procesa uno. Comparar el IPC entre dos versiones con distinto número de instrucciones es engañoso por construcción. La métrica honesta es **elementos por ciclo**, con $N = 10^6$, 0,0318 para la escalar frente a 0,1492 para la vectorial. Un IPC

Tabla 8: *Speedup* por fase del kernel.

N	sum_array	compute_stats	normalize_array
10^3	$6,94\times$	$7,86\times$	$7,75\times$
10^5	$7,05\times$	$8,32\times$	$4,52\times$
10^6	$6,08\times$	$7,65\times$	$4,07\times$
5×10^7	$5,52\times$	$7,21\times$	$3,06\times$

Tabla 9: Ciclos de referencia (TSC) por elemento procesado.

N	Escalar	Vectorial
10^3	26,850	4,404
10^5	25,441	3,425
10^6	24,743	3,670
5×10^7	24,598	4,084

bajo acompañado de muchas menos instrucciones totales es precisamente la firma de un programa limitado por memoria.

Los fallos de caché confirman la optimización no temporal: El tráfico teórico mínimo del algoritmo es de 0,3125 líneas de caché por elemento, cuatro recorridos de lectura más una escritura a cuatro bytes por elemento en líneas de 64 bytes. Con $N = 5 \times 10^7$ la versión vectorial mide 0,3173 fallos por elemento, apenas un 15% por encima de ese mínimo teórico: el kernel accede a memoria de forma prácticamente óptima y no queda margen de mejora en el patrón de acceso. La versión escalar mide 0,5486, un 75% por encima del mínimo, diferencia atribuible al *read for ownership* que `vmovntps` evita en la versión vectorial.

Nota metodológica: Una primera medición con solo 5 repeticiones sobre $N = 5 \times 10^7$ arrojó una razón de instrucciones de $5,95\times$ en lugar de $7,58\times$. La discrepancia se debe a que `perf stat` mide el **proceso completo**, incluida la lectura y escritura de los 191 MB del archivo, que es un costo fijo que no escala con las repeticiones. Al aumentar a 20 repeticiones esa contribución se diluye y la razón converge al valor esperado. El tiempo de sistema reportado por `perf` (0,478s con 5 repeticiones frente a 0,342s con 20, mientras el trabajo útil se cuadruplicaba) confirma la explicación. Por la misma razón, los tiempos medidos bajo `perf` no son comparables con los de la Tabla 7, obtenidos sin instrumentación.

VIII. VERIFICACIÓN CON GDB

La sección 2.4, apartado a, inciso 3 del enunciado exige inspeccionar en tiempo de depuración el contenido de un registro YMM inmediatamente después de una instrucción

Tabla 10: Contadores de hardware, $N = 10^6$, 30 repeticiones.

Métrica	Escalar	Vectorial	Razón
instructions	1 164,97 M	148,08 M	7,87×
cycles	942,96 M	201,02 M	4,69×
IPC	1,235	0,737	—
cache-references	22,41 M	15,95 M	1,41×
cache-misses	13,74 M	9,30 M	1,48×
Tasa de fallos	613 %	583 %	—

Tabla 11: Contadores de hardware, $N = 5 \times 10^7$, 20 repeticiones.

Métrica	Escalar	Vectorial	Razón
instructions	39 714,29 M	5 240,23 M	7,58×
cycles	32 263,41 M	5 861,95 M	5,50×
IPC	1,231	0,894	—
cache-references	772,31 M	499,44 M	1,55×
cache-misses	548,64 M	317,26 M	1,73×
Tasa de fallos	710 %	635 %	—

vaddps, y el contenido de memoria del arreglo de salida tras el bucle de normalización. La sesión se automatizó en tools/sesion_gdb.txt y se ejecuta con:

```
1 $ python3 tools/gen_input.py 16 data/n16.dat random 42
2 $ gdb -x tools/sesion_gdb.txt --args ./bin/norm_vector \
3     data/n16.dat data/o16.dat 1
```

Los puntos de ruptura se colocan sobre **etiquetas del ensamblador** y no sobre números de línea, de modo que el guión sigue siendo válido aunque el archivo se edite. GDB expone las etiquetas locales de NASM con la forma funcion.etiqueta.

VIII-A. Inspección del registro YMM

Con $N = 16$ el bucle vectorial ejecuta exactamente dos iteraciones.

```
1 Breakpoint 1, sum_array.sum_vec_loop ()
2 => 0x555555556568 <sum_array.sum_vec_loop+9>:
3     vaddps %ymm1, %ymm0, %ymm0
4
5 === ITERACION 1: ANTES del vaddps ===
6 $1 = {0, 0, 0, 0, 0, 0, 0, 0} <- ymm0
7 $2 = {54.7912102, -12.2243118, 71.7195816,
8     39.4736061, -81.1645279, 95.1244736,
9     52.2279396, 57.2128601} <- ymm1
10
11 === ITERACION 2: DESPUES del vaddps ===
12 $5 = {-19.5860634, -22.1471252, 45.8791885,
13     124.826599, -52.3915024, 159.676788,
14     40.910778, 2.66060257} <- ymm0
```

Listing 12: Contenido de ymm0 antes y después del vaddps.

La verificación carril por carril confirma el modelo de ejecución. Los elementos 8 a 15 de la entrada son $\{-74, 3773, -9, 9228, -25, 8404, 85, 3530, 28, 7730, 64, 5523, -11, 3172, -54, 5523\}$, de modo que:

$$\text{carril 0 : } 54,7912 + (-74,3773) = -19,5861$$

$$\text{carril 1 : } -12,2243 + (-9,9228) = -22,1471$$

$$\text{carril 2 : } 71,7196 + (-25,8404) = 45,8792$$

y así con los ocho. Cada carril k acumula $\text{arr}[k] + \text{arr}[k+8]$ de forma independiente: el registro **no contiene un resultado sino ocho acumuladores separados**, que es precisamente el modelo SIMD.

La suma de los ocho carriles vale 279,8293, exactamente el valor que el programa reporta como suma total. Esto verifica de forma independiente que la reducción horizontal combina correctamente los carriles.

VIII-B. Inspección de memoria y de la ABI

```
1 === argumentos segun la ABI System V AMD64 ===
2 rdi (puntero in) = 0x55555555b4c0
3 rsi (puntero out) = 0x55555555a480
4 edx (n) = 16
5 $7 = 17.4893284 <- xmm0 = mean
6 $8 = 54.9311981 <- xmm1 = stddev
7 rsi mod 32 = 0 <- alineacion verificada
8
9 === arreglo de salida (x/8fw) ===
10 0x55555555a480: 0.679065466 -0.540924668
11                0.987239599 0.400214761
12 0x55555555a490: -1.79595304 1.41331601
13                0.632402122 0.723150671
```

Listing 13: Argumentos recibidos y arreglo de salida tras el bucle de normalización.

Esta captura verifica **en tiempo de ejecución** tres propiedades afirmadas en la sección IV. Primero, la convención de llamada: los dos punteros y el entero llegaron en rdi, rsi y edx, y los dos flotantes en xmm0 y xmm1, en una cola independiente. Segundo, la alineación: rsi mód 32 = 0 confirma que `aligned_alloc(32, ...)` cumplió y que `vmovaps` es seguro. Tercero, la correctud del cálculo: $(54,7912 - 17,4893)/54,9312 = 0,679065$, que coincide con el primer elemento del arreglo de salida hasta la novena cifra significativa, diferencia atribuible al redondeo de `float32`.

Conviene señalar que las mediciones de tiempo que el *driver* reporta durante una sesión de GDB carecen de sentido, pues los puntos de ruptura detienen el programa dentro de la región medida. Como indica el propio enunciado, GDB es una herramienta de verificación funcional y de inspección de bajo nivel, no de medición de tiempo.

IX. PRECISIÓN NUMÉRICA

IX-A. Cancelación catastrófica en una sola pasada

La identidad $\text{Var}(X) = E[X^2] - E[X]^2$ permitiría calcular la varianza en un único recorrido. Se descartó por **cancelación catastrófica** [4]: la mantisa de 24 bits de `float32` proporciona unos siete dígitos decimales significativos, y cuando la media es grande respecto a la dispersión, $E[X^2]$ y $E[X]^2$ son dos cantidades enormes y casi iguales; al restarlas los dígitos significativos se cancelan y sobrevive únicamente el error de redondeo.

Tabla 12: Varianza de 10^3 valores alrededor de 10^6 con dispersión ± 1 .

Método	Varianza	Error rel.
Referencia (float64)	0,340189	—
Dos pasadas (float32)	0,340219	$8,89 \times 10^{-5}$
Una pasada $E[X^2] - E[X]^2$	-7 077 888	$2,08 \times 10^7$

La versión de una pasada devuelve una varianza **negativa**, matemáticamente imposible. El método de dos pasadas evita el problema porque la resta $x_i - \mu$ se realiza antes de elevar al cuadrado, de modo que nunca se construyen las magnitudes enormes que después habría que cancelar.

IX-B. Error de acumulación secuencial

Una implementación inicial con un único acumulador en `float32` superó la tolerancia exigida a partir de $N = 10^6$. El diagnóstico es directo: al acumular $\sum (x_i - \mu)^2$ sobre 10^6 elementos el acumulador alcanza un valor del orden de $3,3 \times 10^9$, y a esa magnitud el espaciado entre dos `float32` consecutivos es de unos 400 unidades, mientras que cada término que se suma vale aproximadamente 3300. El acumulador se vuelve tan grande respecto al sumando que cada suma pierde bits bajos. El error crece como $O(\sqrt{n}\varepsilon)$ en la práctica [5].

Un hallazgo relevante es que **la versión vectorial resultaba más precisa que la escalar** sin ninguna medida adicional. La razón es que sus ocho carriles constituyen ocho acumuladores independientes, cada uno con $n/8$ términos, lo que equivale a una suma por bloques: con $N = 10^6$ el error relativo de la varianza era de $3,55 \times 10^{-4}$ en la escalar frente a $1,93 \times 10^{-5}$ en la vectorial, unas 18 veces mejor. La vectorización, además de acelerar, mejoraba la exactitud.

Aun así, con $N = 5 \times 10^7$ también la versión vectorial fallaba ($5,53 \times 10^{-3}$), porque cada carril acumulaba $6,25 \times 10^6$ términos.

IX-C. Mitigaciones evaluadas

Se consideraron cuatro alternativas:

1. **Acumuladores parciales** mediante desenrollado del bucle, que reproduce en la versión escalar la ventaja de los carriles.
2. **Suma por pares** (*pairwise summation*), con error $O(\log n \cdot \varepsilon)$; es la que emplea `numpy.sum` internamente.
3. **Suma compensada de Kahan**, con error $O(\varepsilon)$ independiente de n , a costa de tres operaciones adicionales por elemento.
4. **Acumulación en doble precisión**, que resuelve el problema pero contraviene el requisito de que todo el cómputo se realice en `float32`.

Se adoptó la **suma compensada de Kahan**, porque es la única que elimina la dependencia del error con n sin abandonar la precisión simple. La Tabla 13 muestra que alcanza exactamente la misma exactitud que acumular en doble precisión.

Tabla 13: Error relativo de la varianza según la estrategia de acumulación.

Estrategia	$N = 10^6$	$N = 5 \times 10^7$
<code>float32</code> , un acumulador (escalar)	$3,55 \times 10^{-4}$	—
<code>float32</code> , 8 carriles (vectorial)	$1,93 \times 10^{-5}$	$5,53 \times 10^{-3}$
Acumulación en <code>float64</code>	$2,53 \times 10^{-8}$	$2,74 \times 10^{-9}$
Kahan en <code>float32</code>	$2,53 \times 10^{-8}$	$2,74 \times 10^{-9}$

El costo es real y se cuantificó: la compensación de Kahan añade cuatro operaciones por elemento y, más importante, **alarga la cadena de dependencias serial** del bucle, ya que cada iteración necesita la compensación de la anterior. La penalización medida fue de aproximadamente un 38 % en la versión escalar y un 33 % en la vectorial. Como ambas pagan

un costo similar, el *speedup* apenas se altera: se ganó exactitud en cuatro órdenes de magnitud sin modificar la conclusión sobre el beneficio de la vectorización.

IX-D. Interacción entre FMA y la compensación de Kahan

El procesador soporta FMA, y una optimización aparentemente evidente sería fusionar `vmulps` y `vaddps` en un `vfmadd`. Se descartó por una razón de fondo: tras introducir la compensación de Kahan, la resta del término de compensación queda **entre** la multiplicación y la suma, de modo que no son fusionables directamente. Más importante, la demostración de que la compensación de Kahan captura exactamente el error de redondeo depende de que cada operación redondee por separado según IEEE-754; el redondeo único de FMA invalida ese supuesto y puede anular silenciosamente la compensación.

IX-E. Alcance de la compensación

La suma compensada se aplica únicamente a la **pasada de varianza**. `sum_array` y la pasada de la media acumulan sin compensación (un acumulador en la escalar, ocho carriles en la vectorial). Hay dos razones. Primero, los términos de la varianza son todos positivos y el acumulador crece hasta $\sim 10^9$, que es donde se pierden bits; la suma de datos de media cercana a cero se mantiene pequeña. Segundo, un error δ en μ afecta a la varianza solo en segundo orden ($n\delta^2$), por la forma del algoritmo de dos pasadas. La limitación es real: con datos de media grande y N grande la suma sufriría el mismo problema de acumulación.

La compensación de Kahan cubre solo la varianza; la suma y la media usan acumulación simple y podrían degradarse con datos de media grande.

X. CONCLUSIONES Y TRABAJO FUTURO

- La versión vectorizada con AVX2 alcanza un *speedup* de $7,70\times$ con $N = 10^3$ que decrece monótonamente hasta $6,02\times$ con $N = 5 \times 10^7$. El decrecimiento se explica por el descenso del conjunto de trabajo a través de la jerarquía de memoria: con $N = 10^3$ cabe en L1, con $N = 10^5$ en L3, y con $N \geq 10^6$ excede toda la caché.
- **El *speedup* no alcanza el $8\times$ teórico porque el cuello de botella se desplaza de cómputo a ancho de banda de memoria.** La evidencia es la brecha entre dos contadores de hardware: la versión vectorial ejecuta 7,58 a 7,87 veces menos instrucciones —prácticamente el límite del registro YMM— pero consume solo 4,69 a 5,50 veces menos ciclos. Si el límite fuera el cómputo, ambas razones coincidirían.
- El análisis por fases identificó `normalize_array` como el factor limitante: su *speedup* cae hasta $3,06\times$ mientras `compute_stats` se mantiene sobre $7\times$. Es la única fase que escribe y por tanto la que soporta el doble de tráfico de memoria. Aplicar multiplicación por el recíproco y almacenamiento no temporal, seleccionados según el tamaño del arreglo, elevó su *speedup* de 2,00 a $3,06\times$ con $N = 5 \times 10^7$.
- La acumulación secuencial en `float32` con un único acumulador excede la tolerancia exigida a partir de

$N = 10^6$. La vectorización resulta **intrínsecamente más precisa** porque sus ocho carriles actúan como ocho acumuladores independientes, pero no basta en el tamaño mayor. La suma compensada de Kahan reduce el error a $2,74 \times 10^{-9}$ con $N = 5 \times 10^7$, igualando la exactitud de la doble precisión sin abandonar `float32`.

- Los fallos de caché de la versión vectorial (0,3173 por elemento) están a un 15 % del mínimo teórico del algoritmo (0,3125), lo que indica que el patrón de acceso a memoria es prácticamente óptimo y que cualquier mejora adicional debería provenir de reducir el número de recorridos, no de acceder mejor.

- [2] A. Fog, *Instruction Tables: Lists of Instruction Latencies, Throughputs and Micro-operation Breakdowns*, 2025. dirección: https://www.agner.org/optimize/instruction_tables.pdf.
- [3] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 248966-040US, 2024.
- [4] D. Goldberg, «What Every Computer Scientist Should Know About Floating-Point Arithmetic,» *ACM Computing Surveys*, vol. 23, n.º 1, págs. 5-48, 1991. DOI: 10.1145/103162.103163.
- [5] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2.^a ed. Society for Industrial y Applied Mathematics, 2002. DOI: 10.1137/1.9780898718027.

X-A. Limitaciones observadas

El kernel realiza **cuatro recorridos de lectura** del arreglo por ejecución: uno en `sum_array`, dos en `compute_stats` (media y varianza) y uno en `normalize_array`. En el régimen limitado por memoria, ese número es el factor dominante del tiempo total.

La medición en tamaños pequeños presenta una dispersión elevada inherente: con tiempos de kernel del orden de 100 μ s, las interrupciones periódicas del sistema operativo afectan a una fracción apreciable de las muestras. El promedio permanece bien determinado (el error estándar se mantiene por debajo del 2 %) pero la desviación muestral no es un buen indicador de calidad en ese régimen.

Finalmente, todas las mediciones se realizaron sobre una única microarquitectura. Las latencias y *throughputs* de instrucciones como `vdivps`, así como la penalización de transición AVX–SSE, son específicos de Skylake y podrían diferir en otras generaciones.

X-B. Trabajo futuro

- **Fusionar `sum_array` con `compute_stats`** para eliminar un recorrido completo del arreglo. Es la mejora de mayor impacto potencial en el régimen limitado por memoria, dado que reduciría el tráfico en un 20 %.
- **Algoritmo de Welford**, que calcula media y varianza en una sola pasada de forma numéricamente estable. Se descartó aquí porque requiere una división por iteración y su vectorización no es directa, pero eliminaría otro recorrido.
- **Sustituir `vhaddps` por `vshufps` + `vaddps`** en la reducción horizontal, por consistencia con el enfoque ya empleado para mínimo y máximo.
- **Calibración más fina del umbral de almacenamiento no temporal**. Los datos de la Tabla 4 sugieren que el óptimo podría estar por debajo de los 393 216 elementos derivados del tamaño de L3.
- **AVX-512 en hardware** que lo soporte, evaluando el compromiso entre el doble de ancho vectorial y la posible reducción de frecuencia del núcleo.

REFERENCIAS

- [1] x86-64 psABI Project, *System V Application Binary Interface: AMD64 Architecture Processor Supplement*, 2018. dirección: <https://gitlab.com/x86-psABIs/x86-64-ABI>.